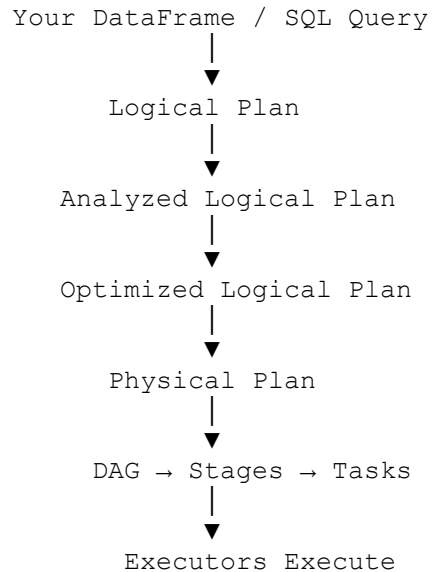


Spark Query Plan

A **Spark Query Plan** is the sequence of steps Spark generates to execute your DataFrame or SQL query efficiently. Before processing any data, Spark analyzes, optimizes, and creates an execution plan.

The query plan goes through **four main stages**.



1. Logical Plan

The **Logical Plan** is created immediately after you write your Spark code.

It describes **what** operations should be performed, without considering how they will be executed.

Example:

```
df.filter(df.salary > 5000).select("name", "salary")
```

Spark creates a logical plan similar to:

```
Project [name, salary]
  |
  Filter salary > 5000
    |
    Scan employees
```

At this stage:

- Spark only records transformations.
 - No optimization is performed.
 - No data is read.
 - This is part of Spark's **lazy evaluation**.
-

2. Analyzed Logical Plan

Spark verifies that the query is valid.

It checks:

- Do the tables exist?
- Do the columns exist?
- Are the data types compatible?
- Are functions valid?

Example:

```
df.select("salary")
```

If salary exists:

```
Project [salary]
  |
Scan employees
```

If you write:

```
df.select("income")
```

Spark throws an error:

```
AnalysisException:
Column 'income' does not exist.
```

3. Optimized Logical Plan

Now the **Catalyst Optimizer** improves the query without changing its result.

Common optimizations include:

Filter Pushdown

Instead of reading all data:

```
Read File  
|  
Filter salary > 5000
```

Spark pushes the filter to the data source:

```
Read only rows where salary > 5000
```

This reduces disk I/O.

Column Pruning

Instead of reading every column:

```
id  
name  
age  
salary  
country  
email  
phone
```

Spark reads only the required columns:

```
name  
salary
```

Constant Folding

Spark evaluates constant expressions before execution.

Instead of:

```
df.filter(df.salary > (3000 + 2000))
```

Spark changes it to:

```
df.filter(df.salary > 5000)
```

Predicate Simplification

Example:

```
salary > 5000 AND salary > 3000
```

becomes

```
salary > 5000
```

Join Reordering

When multiple tables are joined, Spark chooses a more efficient join order.

4. Physical Plan

Spark now decides **how** the query will run.

It chooses algorithms such as:

- Broadcast Hash Join
- Sort Merge Join
- Shuffle Hash Join
- Hash Aggregate
- Sort Aggregate
- File Scan
- Exchange (Shuffle)

Example:

```
HashAggregate
  |
Exchange
  |
Filter
  |
FileScan parquet
```

This is the actual execution plan used by the executors.

Example

```
employees.filter("salary > 5000") \
    .groupBy("department") \
    .count()
```

Logical Plan

```
GroupBy department
|
Filter salary > 5000
|
Scan employees
```

Optimized Logical Plan

```
Read employees
|
Push Filter salary > 5000
|
Aggregate department
```

Physical Plan

```
HashAggregate
|
Exchange (Shuffle)
|
HashAggregate
|
Filter
|
FileScan parquet
```

Spark converts this physical plan into:

```
Physical Plan
|
▼
DAG
|
▼
Stages
|
▼
Tasks
|
▼
Executors
```

Each **task** processes one partition of data.

How to View the Query Plan

explain()

```
df.explain()
```

Shows the physical execution plan.

explain(True)

```
df.explain(True)
```

Displays all stages of the query plan:

- Parsed Logical Plan
- Analyzed Logical Plan
- Optimized Logical Plan
- Physical Plan

Example:

```
employees.filter("salary > 5000") \  
    .groupBy("department") \  
    .count() \  
    .explain(True)
```

Output (simplified):

```
== Parsed Logical Plan ==  
Aggregate [department]  
  +- Filter salary > 5000  
  
== Analyzed Logical Plan ==  
department: string, count: bigint  
  
== Optimized Logical Plan ==  
Aggregate  
  +- Filter salary > 5000  
    +- FileScan parquet  
  
== Physical Plan ==  
HashAggregate  
  +- Exchange  
    +- HashAggregate  
      +- Filter  
        +- FileScan parquet
```

Spark Query Plan Summary

Stage	Purpose
Logical Plan	Records what operations should be performed.
Analyzed Logical Plan	Validates tables, columns, functions, and data types.
Optimized Logical Plan	Uses the Catalyst Optimizer to improve the query (filter pushdown, column pruning, constant folding, join optimization, etc.).
Physical Plan	Chooses the execution algorithms (joins, aggregations, scans, shuffles) and converts the plan into a DAG of stages and tasks for execution by executors.

Interview Answer (Short)

A Spark Query Plan is the execution blueprint Spark creates for a DataFrame or SQL query. It goes through four stages: Logical Plan, Analyzed Logical Plan, Optimized Logical Plan, and Physical Plan. The Catalyst Optimizer improves the logical plan by applying optimizations such as filter pushdown, column pruning, and constant folding. Finally, Spark generates a physical plan, converts it into a DAG of stages and tasks, and executes those tasks on executors.